TECHNICAL ASSESSMENT

Student Management System - Registry Module

Tech Stack	Next.js (App Router) · PostgreSQL · Prisma ORM
Submission	GitHub repository + README
Time Allowed	Within 7 working days
AI Usage	Encouraged - document what you used and how

Overview

Build a focused web application covering the core Registry function of a student management system. You are not building a full platform - you are building the four workflows a Registry Administrator uses every day.

We care more about how you think than how much you build. Make deliberate product decisions, handle edge cases, and document your reasoning. Use AI tools freely - but own the output.

What to Build

1. Student Enrolment

The Registry team needs to add and manage student records.

- Create a student record: full name, email, date of birth, programme, academic year, and enrolment status.
- Auto-generate a unique Student ID (e.g. SMS-2025-0001).
- Enrolment statuses: Enrolled, Deferred, Withdrawn, Completed.
- Search and filter students by name, ID, programme, or status.

2. Fees & Payments

The Registry needs to track what each student owes and what they have paid.

- Assign a fee amount to each student based on their programme.
- Record payment transactions: amount, date, and reference number.
- Show outstanding balance in real time.
- Flag students with an overdue balance on the Registry dashboard.

3. Assessment Submission

Students submit work against assessments created by staff.

- Staff creates an assessment: title, module, and submission deadline.
- Students upload a file (PDF or DOCX) against an open assessment.
- One submission per student per assessment; allow resubmission before the deadline.
- Late submissions are accepted but visually flagged.

4. Marksheet & Results

Staff enter grades; students see results only when published.

- Staff enter a numeric grade (0–100) per student per assessment.
- `Apply a simple classification: Pass ≥ 40, Merit ≥ 60, Distinction ≥ 70.`
- Staff can publish or withhold results per student.
- Students see their marksheet only after it has been published.

What to Submit

- A GitHub repository with all code committed (no zip files).
- A README covering: how to run it locally, your .env variables, and a short section on how you used AI during the build.
- A seed script that loads demo data: at least 5 students, 2 programmes, fees, and sample grades.
- Basic role separation: a Staff view and a Student view (auth optional – a simple role toggle is fine).

How We Will Assess It

We are looking for four things:

Dimension	Weight
Stakeholder understanding – does your data model and UI reflect how a Registry team actually works?	**30%**
Feature intuition – did you handle edge cases (overdue fees, late submissions, withheld results) without being told to?	**30%**
Technical quality – clean schema, working API routes, basic error handling.	**25%**
AI usage – did you use AI effectively, and can you articulate how in your README?	**15%**

Stack & Constraints

- Next.js 14+ with App Router – required.
- PostgreSQL with Prisma – required. Commit your schema.prisma.
- Styling: Tailwind or any component library (Shadcn UI preferred)
- Do not use a different backend framework.
- Do not mock data in useState - a real database is required.
- Use a .env example file; never commit credentials.

Good luck — we look forward to seeing how you think.
PEN Group — Information Technology Team